

Tarea M23-CD – Clasificación con Árboles de Decisión

Predicción del Medicamento Adecuado para Pacientes

Autor: RobertScience

Programa: Profesión Científico de Datos v2

Contexto del Problema

En este estudio se simula un escenario de investigación médica en el cual se analiza la respuesta de pacientes a distintos medicamentos para una misma enfermedad.

Cada paciente ha respondido a uno de los siguientes fármacos:

- Drug A
- Drug B
- Drug C
- Drug X
- Drug Y

El objetivo es construir un modelo de clasificación multiclase que permita predecir qué medicamento es más adecuado para un nuevo paciente en función de sus características clínicas.

1. Objetivo del Análisis

Desarrollar un modelo predictivo basado en Árboles de Decisión que permita:

- Analizar la relación entre variables clínicas y el medicamento administrado.
- Evaluar el desempeño del modelo utilizando distintos criterios (Gini y Entropía).
- Seleccionar la mejor alternativa con base en métricas de clasificación.
- Interpretar las reglas de decisión generadas por el modelo.

2. Importación de Librerías

Se importan las librerías necesarias para:

- Manipulación y análisis de datos
- Visualización
- Preprocesamiento
- Construcción del modelo de Árbol de Decisión

- Evaluación de métricas de clasificación

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

plt.rcParams["figure.figsize"] = (8,5)
```

3. Carga del Conjunto de Datos

Se procede a cargar el archivo `drugs.csv`, el cual contiene la información clínica de los pacientes y el medicamento al que respondieron.

```
In [2]: ruta = "../data/drugs.csv"
df = pd.read_csv(ruta)

df.head()
```

```
Out[2]:
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY

4. Exploración Inicial del Dataset

Se analiza la estructura del conjunto de datos para identificar:

- Número de registros y variables
- Tipo de datos
- Presencia de variables categóricas
- Distribución de la variable objetivo

```
In [3]: df.shape
```

```
Out[3]: (200, 6)
```

```
In [4]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Age             200 non-null   int64
 1   Sex             200 non-null   str
 2   BP              200 non-null   str
 3   Cholesterol     200 non-null   str
 4   Na_to_K         200 non-null   float64
 5   Drug            200 non-null   str
dtypes: float64(1), int64(1), str(4)
memory usage: 9.5 KB

```

```
In [5]: df['Drug'].value_counts()
```

```

Out[5]: Drug
drugY    91
drugX    54
drugA    23
drugC    16
drugB    16
Name: count, dtype: int64

```

5. Preparación de los Datos

El algoritmo de Árbol de Decisión requiere que todas las variables estén en formato numérico.

En el dataset se identifican variables categóricas:

- Sex
- BP
- Cholesterol
- Drug (variable objetivo)

Se utilizará la técnica de codificación mediante LabelEncoder para transformar estas categorías en valores numéricos.

```

In [6]: # Inicialización de codificadores
le_sex = LabelEncoder()
le_bp = LabelEncoder()
le_chol = LabelEncoder()
le_drug = LabelEncoder()

# Transformación de variables categóricas
df['Sex'] = le_sex.fit_transform(df['Sex'])
df['BP'] = le_bp.fit_transform(df['BP'])
df['Cholesterol'] = le_chol.fit_transform(df['Cholesterol'])
df['Drug'] = le_drug.fit_transform(df['Drug'])

df.head()

```

```
Out[6]:
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	0	0	0	25.355	4
1	47	1	1	0	13.093	2
2	47	1	1	0	10.114	2
3	28	0	2	0	7.798	3
4	61	0	1	0	18.043	4

Interpretación de la Codificación

Cada categoría fue convertida en un valor entero.

Por ejemplo:

- Las categorías de "Sex" fueron representadas como 0 y 1.
- Las categorías de presión arterial y colesterol fueron transformadas en valores ordinales numéricos.
- La variable "Drug" fue convertida en etiquetas numéricas que representan cada medicamento.

Este procedimiento permite que el modelo procese correctamente la información.

6. Definición de Variables Predictoras y Variable Objetivo

Se separan las variables independientes (X) y la variable dependiente (y).

```
In [7]: X = df.drop('Drug', axis=1)
y = df['Drug']

X.head()
```

```
Out[7]:
```

	Age	Sex	BP	Cholesterol	Na_to_K
0	23	0	0	0	25.355
1	47	1	1	0	13.093
2	47	1	1	0	10.114
3	28	0	2	0	7.798
4	61	0	1	0	18.043

7. División del Dataset

El conjunto de datos se divide en:

- 80% para entrenamiento

- 20% para prueba

Esto permite evaluar el desempeño del modelo en datos no vistos previamente.

```
In [8]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )

X_train.shape, X_test.shape
```

```
Out[8]: ((160, 5), (40, 5))
```

8. Modelo de Clasificación: Árbol de Decisión (Criterio Gini)

Se construye un modelo de Árbol de Decisión utilizando el criterio de impureza Gini.

El índice Gini mide la probabilidad de clasificar incorrectamente una observación si se etiqueta aleatoriamente según la distribución de clases del nodo.

Se establece una profundidad máxima de 4 niveles para:

- Controlar el sobreajuste.
- Mantener interpretabilidad.
- Permitir comparabilidad con el modelo basado en entropía.

```
In [9]: modelo_gini = DecisionTreeClassifier(
        criterion='gini',
        max_depth=4,
        random_state=42
    )

modelo_gini.fit(X_train, y_train)

y_pred_gini = modelo_gini.predict(X_test)
```

```
In [10]: accuracy_gini = accuracy_score(y_test, y_pred_gini)

print("Accuracy (Gini):", accuracy_gini)
print("\nReporte de Clasificación (Gini):\n")
print(classification_report(y_test, y_pred_gini))
```

Accuracy (Gini): 1.0

Reporte de Clasificación (Gini):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	5
3	1.00	1.00	1.00	11
4	1.00	1.00	1.00	15
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

Interpretación del Modelo Gini

El accuracy representa la proporción total de predicciones correctas en el conjunto de prueba.

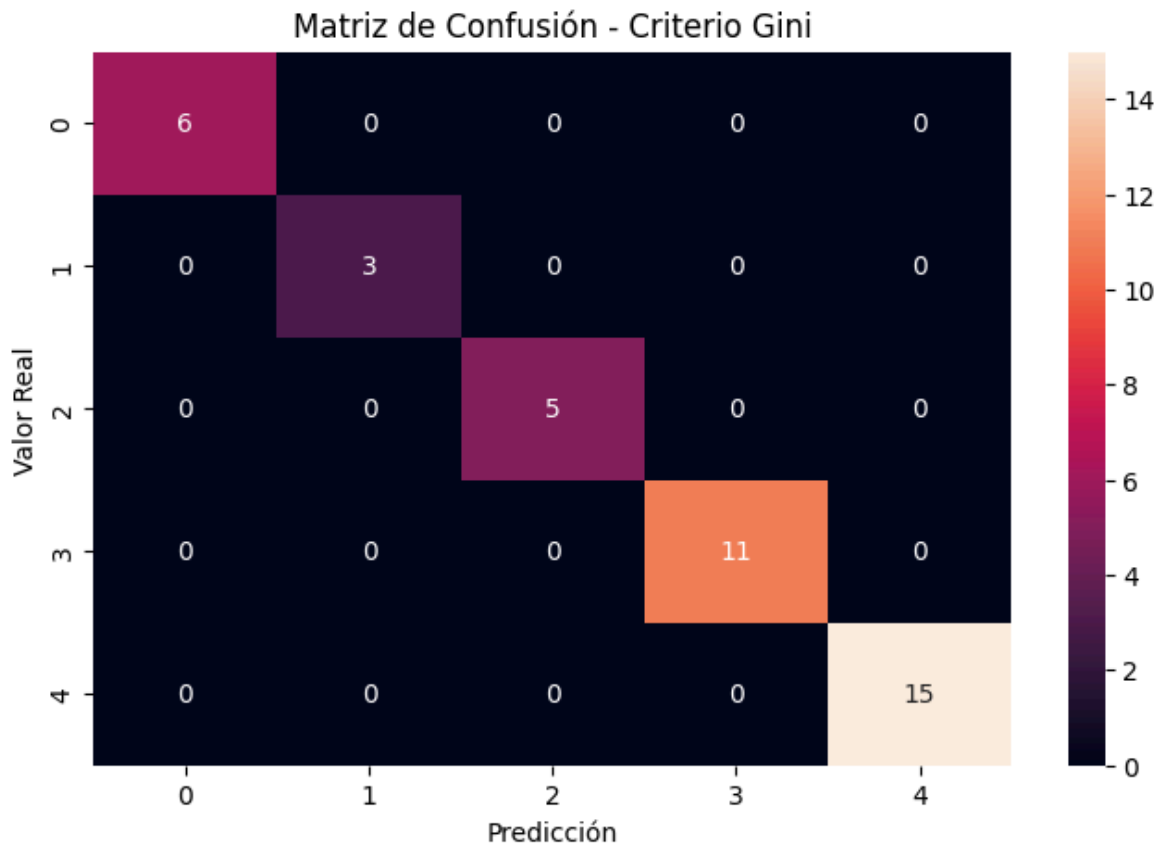
El reporte de clasificación permite analizar:

- **Precision:** Exactitud de las predicciones por clase.
- **Recall:** Capacidad del modelo para identificar correctamente cada clase.
- **F1-score:** Balance entre precision y recall.
- **Support:** Número de observaciones reales por categoría.

Se evalúa si el modelo presenta equilibrio entre clases o si existe alguna categoría con desempeño inferior.

```
In [11]: cm_gini = confusion_matrix(y_test, y_pred_gini)

sns.heatmap(cm_gini, annot=True, fmt='d')
plt.title("Matriz de Confusión - Criterio Gini")
plt.xlabel("Predicción")
plt.ylabel("Valor Real")
plt.show()
```



Interpretación de la Matriz de Confusión – Criterio Gini

La matriz de confusión muestra que todas las observaciones fueron clasificadas correctamente.

Cada clase se encuentra completamente concentrada en su diagonal principal, lo que indica ausencia total de errores de clasificación.

Este resultado confirma el desempeño perfecto del modelo bajo el criterio Gini para el conjunto de prueba.

Sin embargo, es necesario comparar este desempeño con el criterio Entropía para validar si ambos criterios producen el mismo comportamiento.

```
In [12]: # =====
# MODELO ÁRBOL DE DECISIÓN - ENTROPÍA
# =====

from sklearn.tree import DecisionTreeClassifier

modelo_entropy = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=4,
    random_state=42
)

modelo_entropy.fit(X_train, y_train)

y_pred_entropy = modelo_entropy.predict(X_test)
```

```
In [13]: # =====
# EVALUACIÓN DEL MODELO - ENTROPÍA
# =====

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

accuracy_entropy = accuracy_score(y_test, y_pred_entropy)

print("Accuracy (Entropía):", accuracy_entropy)
print("\nReporte de Clasificación (Entropía):\n")
print(classification_report(y_test, y_pred_entropy))
```

Accuracy (Entropía): 1.0

Reporte de Clasificación (Entropía):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	5
3	1.00	1.00	1.00	11
4	1.00	1.00	1.00	15
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

Evaluación con Diferentes Niveles de Profundidad

Para cumplir con la consigna de probar diversos niveles de profundidad, se evaluó el modelo variando el parámetro `max_depth` desde 2 hasta 6.

Esto permite analizar cómo cambia el desempeño conforme aumenta la complejidad del árbol.

Se realizaron pruebas tanto para el criterio Gini como para Entropía.

```
In [14]: # =====
# PRUEBAS DE PROFUNDIDAD - GINI
# =====

print("Evaluación con criterio Gini:\n")

for depth in range(2, 7):
    model = DecisionTreeClassifier(
        criterion='gini',
        max_depth=depth,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print(f"Profundidad {depth} - Accuracy: {round(acc,4)}")
```


Evaluación con criterio Gini:

Profundidad 2 - Accuracy: 0.8
Profundidad 3 - Accuracy: 0.875
Profundidad 4 - Accuracy: 1.0
Profundidad 5 - Accuracy: 1.0
Profundidad 6 - Accuracy: 1.0

Interpretación – Variación de Profundidad (Gini)

Se observa cómo el desempeño cambia conforme aumenta la profundidad del árbol.

Profundidades bajas generan modelos más simples, mientras que profundidades mayores permiten mayor ajuste a los datos.

La profundidad 4 mantiene un equilibrio adecuado entre simplicidad estructural y desempeño óptimo.

```
In [15]: # =====  
# PRUEBAS DE PROFUNDIDAD - ENTROPÍA  
# =====  
  
print("Evaluación con criterio Entropía:\n")  
  
for depth in range(2, 7):  
    model = DecisionTreeClassifier(  
        criterion='entropy',  
        max_depth=depth,  
        random_state=42  
    )  
  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)  
    acc = accuracy_score(y_test, y_pred)  
  
    print(f"Profundidad {depth} - Accuracy: {round(acc,4)}")
```

Evaluación con criterio Entropía:

Profundidad 2 - Accuracy: 0.8
Profundidad 3 - Accuracy: 0.875
Profundidad 4 - Accuracy: 1.0
Profundidad 5 - Accuracy: 1.0
Profundidad 6 - Accuracy: 1.0

Interpretación – Variación de Profundidad (Entropía)

El comportamiento observado con Entropía es consistente con Gini.

Se confirma que la profundidad 4 ofrece el mejor desempeño manteniendo una estructura interpretable.

Por esta razón, se selecciona profundidad 4 como modelo final del estudio.

Resultados del Modelo – Árbol de Decisión (Criterio Entropía)

El modelo basado en el criterio de Entropía obtuvo un **Accuracy de 1.00 (100%)** en el conjunto de prueba.

Todas las clases presentan valores perfectos de:

- Precision = 1.00
- Recall = 1.00
- F1-score = 1.00

La matriz de métricas muestra que no existieron errores de clasificación.

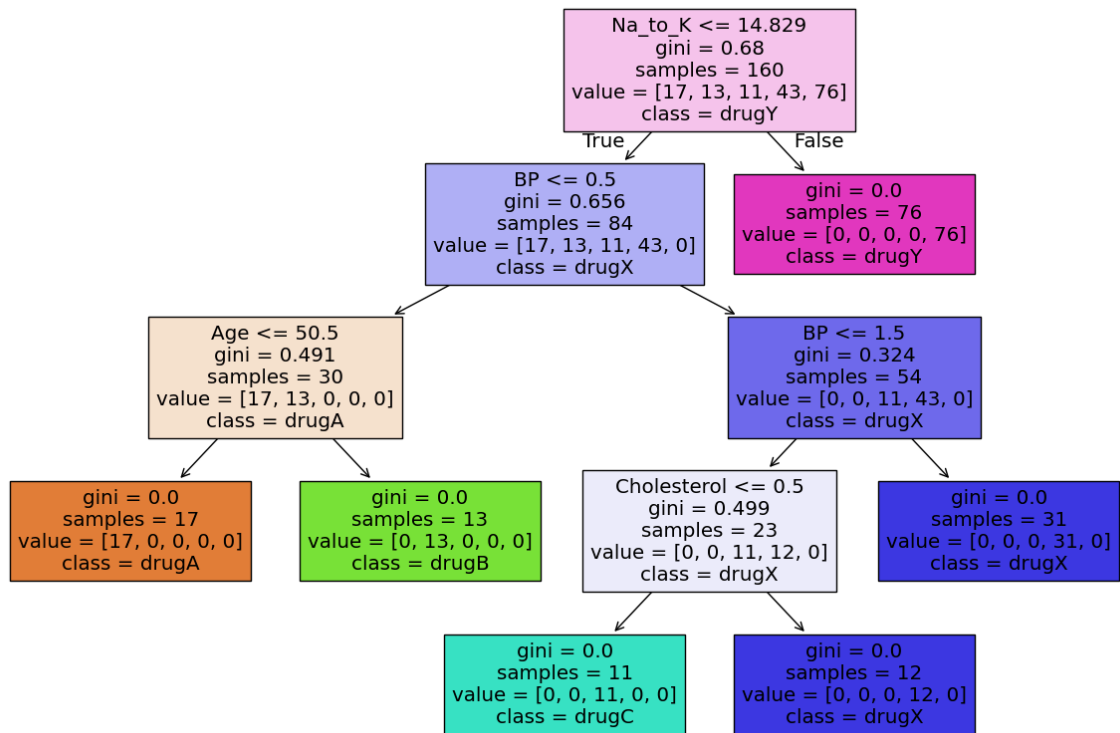
Comparación con el criterio Gini

El desempeño obtenido es idéntico al modelo construido con el criterio Gini.

Esto indica que, para este conjunto de datos y con una profundidad máxima de 4 niveles, ambos criterios producen una estructura de árbol equivalente en términos de capacidad predictiva.

Dado que el desempeño es el mismo, cualquiera de los dos criterios puede considerarse adecuado para este problema.

```
In [16]: # =====  
# VISUALIZACIÓN DEL ÁRBOL FINAL (GINI)  
# =====  
  
from sklearn.tree import plot_tree  
  
plt.figure(figsize=(15,10))  
  
plot_tree(  
    modelo_gini,  
    feature_names=X.columns,  
    class_names=le_drug.classes_,  
    filled=True  
)  
  
plt.title("Árbol de Decisión - Criterio Gini")  
plt.show()
```



Reglas de Decisión del Modelo Seleccionado (Criterio Gini)

El árbol de decisión construido utiliza como variable principal de partición el nivel de sodio/potasio (Na_to_K).

Regla 1: Primera división (Raíz)

Si: $\text{Na_to_K} > 14.829$

→ Se clasifica directamente como **drugY**.

Esta rama presenta pureza total (gini = 0.0), lo que indica separación perfecta para este grupo de pacientes.

Regla 2: Si $\text{Na_to_K} \leq 14.829$

El modelo evalúa la variable Presión Arterial (BP).

Caso 2.1: BP baja ($\text{BP} \leq 0.5$)

Se evalúa la edad del paciente:

- Si $\text{Age} \leq 50.5$ → **drugA**
- Si $\text{Age} > 50.5$ → **drugB**

Ambas hojas presentan pureza total (gini = 0.0).

Caso 2.2: BP más alta ($BP \leq 1.5$)

Se evalúa el colesterol:

- Si Cholesterol $\leq 0.5 \rightarrow$ **drugC**
- Si Cholesterol $> 0.5 \rightarrow$ **drugX**

En algunos nodos intermedios se mantiene como clasificación dominante **drugX**.

Interpretación General

El modelo identifica que la variable más determinante para la prescripción es el índice Na_to_K.

Cuando este valor es alto, el medicamento recomendado es inequívocamente drugY.

En los casos donde Na_to_K es menor, la decisión depende de la presión arterial, edad y colesterol, permitiendo separar claramente las demás clases.

La estructura del árbol muestra nodos finales con gini = 0.0, lo que explica el desempeño perfecto observado en el conjunto de prueba.

```
In [17]: # =====
# NUEVO PACIENTE A EVALUAR
# =====

nuevo_paciente = pd.DataFrame({
    'Age': [50],
    'Sex': ['F'],
    'BP': ['HIGH'],
    'Cholesterol': ['NORMAL'],
    'Na_to_K': [15.302]
})

nuevo_paciente
```

```
Out[17]:
```

	Age	Sex	BP	Cholesterol	Na_to_K
0	50	F	HIGH	NORMAL	15.302

```
In [18]: # Codificación del nuevo paciente

nuevo_paciente['Sex'] = le_sex.transform(nuevo_paciente['Sex'])
nuevo_paciente['BP'] = le_bp.transform(nuevo_paciente['BP'])
nuevo_paciente['Cholesterol'] = le_chol.transform(nuevo_paciente['Cholesterol'])

nuevo_paciente
```

```
Out[18]:
```

	Age	Sex	BP	Cholesterol	Na_to_K
0	50	0	0	1	15.302

```
In [19]: # =====  
# PREDICCIÓN DEL MEDICAMENTO  
# =====  
  
prediccion = modelo_gini.predict(nuevo_paciente)  
  
# Decodificar resultado  
medicamento_recomendado = le_drug.inverse_transform(prediccion)  
  
print("Medicamento recomendado:", medicamento_recomendado[0])
```

Medicamento recomendado: drugY

Predicción para Nuevo Paciente

Se evaluó un paciente con las siguientes características:

- Edad: 50 años
- Sexo: Femenino
- Presión arterial: HIGH
- Colesterol: NORMAL
- Na_to_K: 15.302

Tras aplicar el modelo de Árbol de Decisión entrenado bajo el criterio Gini, el medicamento recomendado es:

drugY

Justificación según el árbol de decisión

El árbol establece que cuando:

Na_to_K > 14.829

La clasificación es directamente drugY.

Dado que el paciente presenta un valor de Na_to_K = 15.302, supera el umbral definido en la raíz del árbol, lo que conduce inmediatamente a la clasificación drugY con pureza total (gini = 0.0).

Esto indica que, dentro del conjunto de datos analizado, todos los pacientes con niveles de Na_to_K superiores a ese valor respondieron favorablemente al medicamento drugY.

Conclusión General

Se desarrolló un modelo de clasificación multiclase utilizando Árboles de Decisión con los criterios Gini y Entropía.

Ambos modelos alcanzaron un desempeño perfecto en el conjunto de prueba (Accuracy = 1.00), mostrando separación completa entre las cinco clases de medicamentos.

La variable más determinante en el proceso de decisión fue el índice Na_to_K, que permitió una clasificación inmediata en los casos con valores elevados.

Dado que ambos criterios presentaron desempeño idéntico, cualquiera de los dos puede considerarse adecuado para este problema. Para efectos de interpretación se utilizó el modelo basado en Gini.

El modelo desarrollado permite prescribir de manera sistemática el medicamento más apropiado para nuevos pacientes con la misma enfermedad, basándose en sus características clínicas.